

数据库系统第二章—关系代数详细学习笔记

中南大学计算机学院 · 邓磊 · 2026 春

2026 年 6 月 15 日

源课件：数据库系统 PPT-ch2_1 5 关系代数.pdf (共 97 页)

整理涵盖页面：p.1–p.94 (p.95–97 为结尾空白页)

目录

1	关系数据库结构	2
1.1	数据模型回顾	2
1.2	关系的基本定义	2
1.3	关系实例的术语	2
1.4	关系的主要特性 (考点)	2
1.5	码/键 (Keys)	2
1.5.1	超码 (Super Key, SK)	2
1.5.2	候选码 (Candidate Key, CK)	3
1.5.3	主码 (Primary Key, PK) 与备选码 (Alternate Key, AK)	3
1.5.4	外码 (Foreign Key, FK)	3
1.6	完整性约束 (Integrity Constraints)	3
1.6.1	实体完整性 (Entity Integrity)	3
1.6.2	参照完整性 (Referential Integrity)	3
1.6.3	用户定义的完整性 (Enterprise Constraints)	3
2	基本关系代数操作	4
2.1	选择 (Selection, σ)	4
2.2	投影 (Projection, π)	4
2.3	并 (Union, $\cup$)	4
2.4	集合差 (Set Difference, $-$)	4
2.5	笛卡尔积 (Cartesian Product, $\times$)	5
2.6	更名 (Rename, ρ)	5
2.7	关系代数表达式定义 (形式化)	5
3	附加关系代数操作	6
3.1	集合交 (Set Intersection, $\cap$)	6
3.2	自然连接 (Natural Join, $\bowtie$)	6

3.3	除运算 (Division, $\div$)	6
3.4	赋值运算 (Assignment, $\leftarrow$)	7
4	扩展关系代数操作	8
4.1	广义投影 (Generalized Projection)	8
4.2	聚集函数 (Aggregate Functions)	8
4.3	外连接 (Outer Join)	8
4.3.1	左外连接 (Left Outer Join,)	9
4.3.2	右外连接 (Right Outer Join,)	9
4.3.3	全外连接 (Full Outer Join,)	9
4.4	空值 (Null Values) 处理规则	9
5	数据库修改	10
5.1	删除 (Deletion)	10
5.2	插入 (Insertion)	10
5.3	更新 (Updating)	10
6	考试重点与综合技巧	12
6.1	关系代数表达式书写要点	12
6.2	关键等价转换 (考点)	12
6.3	常考关系模式 (银行示例)	12
6.4	常考关系模式 (学校示例)	12
6.5	典型考题类型	12
7	附录: 符号速查表	14
7.1	完整 SQL 连接类型速查	14

1 关系数据库结构

1.1 数据模型回顾

- **数据模型 (Data Model)**: 描述数据、数据间联系、数据语义以及约束的工具集合。由三部分组成: 数据结构 (Data Structure)、数据操纵 (Data Manipulation)、完整性约束 (Integrity Constraints)。[p.4]
- 关系模型是一种数据模型, 具备上述三要素。[p.4]
- 关系模型的提出者: E.F. Codd, 1970 年《A Relational Model of Data for Large Shared Data Banks》; 1972 年提出关系代数与关系演算概念, 为 SQL 奠定了基础。[p.5]

1.2 关系的基本定义

- **关系/Relation**: 给定域 $D_1, D_2, \dots, D_n$, 关系 r 是笛卡尔积 $D_1 \times D_2 \times \dots \times D_n$ 的一个子集。[p.9]
- 关系即一张二维表 (Table), 包含行和列。一个关系由 n -元组 $(a_1, a_2, \dots, a_n)$ 构成, 其中 $a_i \in D_i$ 。[p.9]
- **关系模式 (Relation Schema)**: $R(A_1, A_2, \dots, A_n)$, 其中 A_i 为属性名。对结构的描述 (型), 关系实例是值。[p.10]
- 完整关系模式: $R(U, D, \text{Dom}, F)$ (关系名、属性集、域、属性到域的映射、依赖)。[p.10]

1.3 关系实例的术语

- **元组 (Tuple)**: 关系中的行/记录。[p.11]
- **度/目 (Degree/Arity)**: 属性的个数。[p.11]
- **基数 (Cardinality)**: 元组的数目。[p.11]
- **关系数据库 (Relational Database)**: 一组不重名的规范化关系的集合。[p.13]

1.4 关系的主要特性 (考点)

所有关系必须满足以下性质: [p.15]

1. 关系名不重复 (Distinct relation name)
2. 单元格不可再分 (Every cell is an atomic/single value) ——满足第一范式 1NF
3. 属性名不重复 (Distinct attribute name)
4. 属性值来自同一域 (Attribute values from the same domain)
5. 元组不重复 (No duplicate tuples)
6. 列的顺序无所谓 (Order of attributes has no significance)
7. 行的顺序无所谓 (理论上; 实际可能影响访问效率) (Order of tuples has no significance)

1.5 码/键 (Keys)

1.5.1 超码 (Super Key, SK)

一个或多个属性的集合, 其值可以唯一标识一个元组。[p.17]

- 若 $K \subseteq R$, 且 K 的值足以唯一标识 $r(R)$ 中的每个元组, 则 K 是超码。

- 例：{学号, 课程号}、{学号, 课程号, 姓名} 都是超码。

1.5.2 候选码 (Candidate Key, CK)

超码的任意子集不再是超码（即最小超码 / Minimal SK）。 [p.18]

- 例：{学号, 课程号}、{校园卡号, 课程号} 是候选码。

1.5.3 主码 (Primary Key, PK) 与备选码 (Alternate Key, AK)

- **PK**: 被数据库设计者选中, 用于区分元组的候选码。一个关系只能有一个 PK。 [p.19]
- **AK**: 剩余的候选码。 [p.19]
- 选择 PK 的原则: 选值不变或很少变化的属性, 单属性 PK 优于复合 PK, 整数优于长字符串。 [p.20]

1.5.4 外码 (Foreign Key, FK)

一个关系（参照关系）中的属性是另一关系（被参照关系）的主码, 则该属性为外码。 [p.21]

- 外码取值规则: 取被参照关系主码值, 或取空值 null。
- 例: stud 表中的「课程号」是外码, 参照 course 表的主码「课程号」。

1.6 完整性约束 (Integrity Constraints)

1.6.1 实体完整性 (Entity Integrity)

- 候选码的值必须唯一 (Keys must be unique)。 [p.22]
- 基本表主码的诸属性均不能为空值 (No attribute of a PK can be null)。 [p.22]
- 空值 Null: 表示属性值当前未知或对该元组不适用。 [p.22]

1.6.2 参照完整性 (Referential Integrity)

- 若外码存在, 则其值必须匹配被参照关系中某个元组的候选码值, 或全部为空。 [p.23]
- 外码取参照码值或取空值。

1.6.3 用户定义的完整性 (Enterprise Constraints)

- 用户或 DBA 自定义的附加规则, 如 `balance > 0`。 [p.23]

2 基本关系代数操作

关系代数运算是以一个或两个关系为输入，产生一个新关系为输出的运算。 [p.28]

六大基本运算： [p.27]

σ (选择) | π (投影) | $\cup$ (并) | $-$ (差) | $\times$ (笛卡尔积) | ρ (更名)

2.1 选择 (Selection, σ)

- 定义：按条件选择元组，形成新关系。 [p.29]
- 符号： $\sigma_p(r) = \{t \mid t \in r \text{ and } p(t)\}$
- p 是选择谓词 (Selection Predicate)，由 $\wedge$ (and)、 $\vee$ (or)、 $\neg$ (not) 连接各项构成。
- 每项形式： $\langle \text{属性} \rangle \text{op} \langle \text{属性} \rangle$ 或 $\langle \text{常量} \rangle$ ，其中 $\text{op} \in \{=, \neq, >, \geq, <, \leq\}$ 。 [p.29]
- 作用：选出满足条件的元组（水平分割 / 行选择）。
- 示例 1： $\sigma_{A=B \wedge D>5}(r)$ —— 选出 $A=B$ 且 $D>5$ 的元组。 [p.30]
- 示例 2： $\sigma_{\text{branch_name}=\text{“中行”}}(\text{account})$ —— 找出中行的所有账户。 [p.31]
- 示例 3： $\sigma_{\text{课程号}=02 \wedge \text{成绩}<90}(\text{stud})$ —— 选修 02 号课程且成绩 <90 的学生。 [p.31]

2.2 投影 (Projection, π)

- 定义：选择指定的列，去掉重复行，形成新关系。 [p.32]
- 符号： $\pi_{A_1, A_2, \dots, A_k}(r)$
- 结果包含 k 列，删除未列出的列，自动去重（因为关系是集合）。 [p.32–33]
- 示例 1： $\pi_{A, C}(r)$ —— 只保留 A 列和 C 列，自动去除重复行。 [p.33]
- 示例 2： $\pi_{\text{account_number}, \text{balance}}(\text{account})$ —— 去掉 branch_name 属性。 [p.32]
- 示例 3： $\pi_{\text{学号}, \text{课程号}, \text{成绩}}(\text{stud})$ —— 投影出选课核心信息。 [p.34]
- 何时用投影：只需要一部分列时、做查询的最终结果显示时、在做集合运算前使两个关系兼容时。

2.3 并 (Union, $\cup$)

- 定义： $r \cup s = \{t \mid t \in r \text{ or } t \in s\}$ [p.35]
- 前提条件： [p.35]
 1. r 和 s 必须同度（arity 相同，即属性个数相同）
 2. 对应属性的域必须相容（compatible domains）
- 示例 1： $r \cup s$ （并集，自动去重）。 [p.36]
- 示例 2： $\pi_{\text{customer_name}}(\text{borrower}) \cup \pi_{\text{customer_name}}(\text{depositor})$ —— 找出有贷款或有存款的所有客户。 [p.36]
- 注意：结果自动去重。

2.4 集合差 (Set Difference, $-$)

- 定义： $r - s = \{t \mid t \in r \text{ and } t \notin s\}$ [p.37]
- 前提条件：与并运算相同，需同度且域相容。 [p.37]
- 示例： $r - s = \{\text{在 } r \text{ 中但不在 } s \text{ 中的元组}\}$ 。 [p.38]

- **典型应用：**查找”在 A 中有但在 B 中没有”的查询。如：找出在中行有贷款但在任何银行没有存款的客户。 [p.46]

2.5 笛卡尔积 (Cartesian Product, $\times$)

- **定义：** $r \times s = \{tq \mid t \in r \text{ and } q \in s\}$ [p.39]
- **前提：**假设 $r(R)$ 和 $s(S)$ 的属性不相交 ($R \cap S = \emptyset$)。若有同名属性，必须先更名。 [p.39]
- **结果模式：** $R \cup S$ (所有属性的并集)。
- **示例：** $r \times s$ 产生 $|r| \times |s|$ 行。 [p.40]
- **注意：**笛卡尔积本身意义不大，通常与选择组合使用 (如 $\sigma_{A=C}(r \times s)$)，这是连接的基础。 [p.41]
- **查询示例 (多步复合)：** [p.46]

找出在”中行”有贷款的所有客户的名字：

$$\pi_{\text{customer_name}}(\sigma_{\text{branch_name}=\text{“中行”}}(\sigma_{\text{borrower.loan_number}=\text{loan.loan_number}}(\text{borrower} \times \text{loan})))$$

或等价写法：

$$\pi_{\text{customer_name}}(\sigma_{\text{branch_name}=\text{“中行”} \wedge \text{borrower.loan_number}=\text{loan.loan_number}}(\text{borrower} \times \text{loan}))$$

或 (先缩小再笛卡尔积)：

$$\pi_{\text{customer_name}}(\sigma_{\text{loan.loan_number}=\text{borrower.loan_number}}(\sigma_{\text{branch_name}=\text{“中行”}}(\text{loan}) \times \text{borrower}))$$

2.6 更名 (Rename, ρ)

- **定义：**给关系代数表达式的结果命名，以便引用。 [p.42]
- **符号 1：** $\rho_x(E)$ ——返回表达式 E 的结果并命名为 x 。
- **符号 2：** $\rho_{x(A_1, A_2, \dots, A_n)}(E)$ ——同时重命名属性和关系名。 [p.42]
- **应用场景：** [p.42]
 - 需要引用中间结果
 - 笛卡尔积中避免同名属性冲突
 - 同一个关系需要在查询中出现两次 (如比较同一关系的不同元组)
- **典型示例——找最大余额：** [p.47]

$$\pi_{\text{balance}}(\text{account}) - \pi_{\text{account.balance}}(\sigma_{\text{account.balance} < d.\text{balance}}(\text{account} \times \rho_d(\text{account})))$$

策略：先找出所有非最大余额 (比某个其他余额小的)，再用全部余额减去非最大余额。

2.7 关系代数表达式定义 (形式化)

- **基本表达式：**数据库中的关系、常量关系。 [p.48]
- 若 E_1, E_2 是表达式，则 $\sigma_p(E_1)$ 、 $\pi_s(E_1)$ 、 $E_1 \cup E_2$ 、 $E_1 - E_2$ 、 $E_1 \times E_2$ 、 $\rho_x(E_1)$ 都是表达式。 [p.48]

3 附加关系代数操作

附加运算不增加关系代数的表达能力，但简化常见查询。 [p.55]

3.1 集合交 (Set Intersection, $\cap$)

- 定义: $r \cap s = \{t \mid t \in r \text{ and } t \in s\}$ [p.56]
- 前提: r 和 s 同度、域相容。
- 等价表达式: $r \cap s = r - (r - s)$ 。 [p.56]
- 示例: 找出既有贷款又有存款的客户—— $\pi_{\text{customer_name}}(\text{borrower}) \cap \pi_{\text{customer_name}}(\text{depositor})$ 。 [p.64]

3.2 自然连接 (Natural Join, $\bowtie$)

- 定义: $r \bowtie s$ —— 在 $R \cup S$ 模式上的关系，对每一对元组 $t_r \in r$ 和 $t_s \in s$ ，若它们在 $R \cap S$ （公共属性）上取值相同，则将合并后的元组加入结果。 [p.58]
- 等价表达式: [p.58]

$$r \bowtie s = \pi_{r.A, r.B, r.C, r.D, s.E} (\sigma_{r.B=s.B \wedge r.D=s.D} (r \times s))$$

其中 $r = (A, B, C, D)$, $s = (E, B, D)$ 。

- 示例 1: $r \bowtie s$ （自动匹配公共列 B 和 D）。 [p.59]
- 示例 2: 找出有贷款且知道贷款金额的客户: [p.64]

$$\pi_{\text{customer_name}, \text{loan_number}, \text{amount}}(\text{borrower} \bowtie \text{loan})$$

- 示例 3: 查 SSD7 课程的学生学号和成绩。 [p.69]

$$\pi_{\text{学号}, \text{成绩}}(\sigma_{\text{课程名} = \text{"SSD7"}}(c \bowtie sc))$$

- 自然连接是最常用的连接方式，自动在同名属性上做等值连接并去除重复列。

3.3 除运算 (Division, $\div$)

- 定义: 设 r 模式为 $R = (A_1, \dots, A_m, B_1, \dots, B_n)$, s 模式为 $S = (B_1, \dots, B_n)$, 则: [p.60]

$$r \div s = \{t \mid t \in \pi_{R-S}(r) \wedge \forall u \in s (tu \in r)\}$$

- 语义: 结果模式 $R - S$, 包含所有与 s 中每个元组都组合出现在 r 中的 t 值。 [p.60]
- 适用场景: 含有“for all”（对所有的...）语义的查询。 [p.60]
- 示例 1: [p.61]

设 $r(A, B)$ 和 $s(B)$, $r \div s$ 返回与 s 中所有 B 值都配对的 A 值。

- 示例 2: 找出选修了 1 号和 3 号课程的学生学号。 [p.63]

传统方法: $\pi_{\text{sno}}(\sigma_{\text{Cno}=1}(sc)) \cap \pi_{\text{sno}}(\sigma_{\text{Cno}=3}(sc))$

除法方法: 建立临时关系 $T(\text{Cno}) = \{1, 3\}$, 然后 $\pi_{\text{Sno}, \text{Cno}}(SC) \div T$

- 示例 3: 找出在麓山南路支行和铁道支行都有存款的客户。 [p.65]

$$\pi_{\text{customer_name}, \text{branch_name}}(\text{depositor} \bowtie \text{account}) \div \rho_{\text{temp}}(\text{branch_name}) (\{ \text{"麓山南路支行"}, \text{"铁道支行"} \})$$

- 示例 4: 找出在长沙所有支行都有账户的客户。 [p.67]

$$\pi_{\text{customer_name, branch_name}}(\text{depositor} \bowtie \text{account}) \div \pi_{\text{branch_name}}(\sigma_{\text{branch_city}=\text{"ChangSha"}}(\text{branch}))$$

3.4 赋值运算 (Assignment, $\leftarrow$)

- 用 $\leftarrow$ 将表达式结果赋值给临时关系变量。 [p.55]
- 使复杂查询可以分步书写, 提高可读性。

4 扩展关系代数操作

扩展操作真正增加了关系代数的表达能力。

[p.74]

4.1 广义投影 (Generalized Projection)

- 定义: $\pi_{F_1, F_2, \dots, F_n}(E)$, 其中每个 F_i 是涉及常量和 E 模式中属性的算术表达式。

[p.75]

- 示例:

[p.75]

$$\pi_{\text{customer_name}, \text{limit} - \text{credit_balance}}(\text{credit_info})$$

结果中匿名列需要用 AS 命名, 如 `limit - credit_balance AS credit_avail`。

4.2 聚集函数 (Aggregate Functions)

- 五种聚集函数:

[p.76]

- avg: 平均值 (average value)
- min: 最小值 (minimum value)
- max: 最大值 (maximum value)
- sum: 求和 (sum of values)
- count: 计数 (number of values)

- 语法:

[p.76]

$$G_1, G_2, \dots, G_n \mathcal{G}_{F_1(A_1), F_2(A_2), \dots, F_n(A_n)}(E)$$

其中:

- E 是关系代数表达式
- G_i 是分组属性 (为空则全关系分一组)
- F_i 是聚集函数, A_i 是属性名
- G_i 和 $F_i(A_i)$ 都出现在结果关系中

- 示例——按分行统计余额总和:

[p.78]

$$\text{branch_name} \mathcal{G}_{\text{sum}(\text{balance})}(\text{account})$$

- 结果命名: 聚集结果默认无名, 可用 rename 命名, 如:

[p.79]

$$\text{branch_name} \mathcal{G}_{\text{sum}(\text{balance})} \text{ AS } \text{sum_balance}(\text{account})$$

- 示例——按性别统计平均年龄:

[p.86]

$$\text{性别} \mathcal{G}_{\text{avg}(\text{年龄})} \text{ AS } \text{avg_age}(\text{stud})$$

4.3 外连接 (Outer Join)

- 动机: 避免自然连接丢失信息 (某关系中元组在另一关系中无匹配时被丢弃)。

[p.80]

- 三种外连接 (基于 `loan ⋈ borrower` 示例):

[p.81–83]

4.3.1 左外连接 (Left Outer Join,)

- 保留左关系所有元组，右关系无匹配时填充 null。 [p.82]
- 例：loan 左外连接 borrower ——保留所有贷款记录，没借款人的填 null。

4.3.2 右外连接 (Right Outer Join,)

- 保留右关系所有元组，左关系无匹配时填充 null。 [p.83]
- 例：loan 右外连接 borrower ——保留所有借款人，没贷款记录的填 null。

4.3.3 全外连接 (Full Outer Join,)

- 保留左右关系全部元组，无匹配则对应侧填 null。 [p.83]

4.4 空值 (Null Values) 处理规则

- null 表示未知值或值不存在。 [p.84]
- 任何含 null 的算术表达式结果为 null。 [p.84]
- 聚集函数忽略 null 值（与 SQL 一致）。 [p.84]
- 去重和分组中，null 被视为相同值（与 SQL 一致）。 [p.84]
- **三值逻辑 (Three-Valued Logic):** 涉及 null 的比较返回 **unknown**（非 true/false）。 [p.85]
 - OR: (unknown or true) = true, (unknown or false) = unknown
 - AND: (true and unknown) = unknown, (false and unknown) = false
 - NOT: (not unknown) = unknown
- 选择谓词中，若结果 unknown 则视为 false。 [p.85]
- 连接：两个 null 被视为不同（不等）。 [p.85]
- 投影/并/交：两个 null 被视为相同。 [p.85]

5 数据库修改

所有修改操作均使用赋值运算符表达。

[p.88]

5.1 删除 (Deletion)

- 语法: $r \leftarrow r - E$, 其中 E 是关系代数查询。 [p.89]
- 注意: 只能删除整个元组, 不能只删除某个属性的值。 [p.89]
- 示例 1: 删除 Perryridge 分行的所有账户记录。 [p.90]

$$\text{account} \leftarrow \text{account} - \sigma_{\text{branch_name} = \text{"Perryridge"}}(\text{account})$$

- 示例 2: 删除金额在 0-50 之间的贷款。 [p.90]

$$\text{loan} \leftarrow \text{loan} - \sigma_{\text{amount} \geq 0 \wedge \text{amount} \leq 50}(\text{loan})$$

- 示例 3: 删除位于 Needham 的分行的所有账户 (级联删除, 涉及多表)。[p.90] 需分步进行, 维护参照完整性:

1. $r_1 \leftarrow \sigma_{\text{branch_city} = \text{"Needham"}}(\text{account} \bowtie \text{branch})$
2. $r_2 \leftarrow \pi_{\text{account_number}, \text{branch_name}, \text{balance}}(r_1)$
3. $r_3 \leftarrow \pi_{\text{customer_name}, \text{account_number}}(r_2 \bowtie \text{depositor})$
4. $\text{account} \leftarrow \text{account} - r_2$
5. $\text{depositor} \leftarrow \text{depositor} - r_3$

5.2 插入 (Insertion)

- 语法: $r \leftarrow r \cup E$ [p.91]
- E 可以是常量关系 (单元组插入) 或查询结果 (批量插入)。
- 示例 1: 插入新账户——Smith 在 Perryridge 有账户 A-973, 余额 \$1200。 [p.92]

$$\begin{aligned} \text{account} &\leftarrow \text{account} \cup \{(\text{"A-973"}, \text{"Perryridge"}, 1200)\} \\ \text{depositor} &\leftarrow \text{depositor} \cup \{(\text{"Smith"}, \text{"A-973"})\} \end{aligned}$$

- 示例 2: 给 Perryridge 分行所有贷款客户赠送 \$200 储蓄账户。 [p.92]

$$\begin{aligned} r_1 &\leftarrow \sigma_{\text{branch_name} = \text{"Perryridge"}}(\text{borrower} \bowtie \text{loan}) \\ \text{account} &\leftarrow \text{account} \cup \pi_{\text{loan_number}, \text{branch_name}, 200}(r_1) \\ \text{depositor} &\leftarrow \text{depositor} \cup \pi_{\text{customer_name}, \text{loan_number}}(r_1) \end{aligned}$$

5.3 更新 (Updating)

- 语法: 使用广义投影实现: $r \leftarrow \pi_{F_1, F_2, \dots, F_n}(r)$ [p.93]
- F_i : 若第 i 个属性不更新, 则为该属性本身; 若更新, 则为包含常量和属性的表达式。 [p.93]
- 示例 1: 所有余额增加 5%。 [p.94]

$$\text{account} \leftarrow \pi_{\text{account_number}, \text{branch_name}, \text{balance} * 1.05}(\text{account})$$

- 示例 2: 余额超过 \$10,000 的账户涨 6%，其余涨 5%。

[p.94]

$$\text{account} \leftarrow \pi_{\text{account_number}, \text{branch_name}, \text{balance} * 1.06} (\sigma_{\text{balance} > 10000}(\text{account})) \cup \\ \pi_{\text{account_number}, \text{branch_name}, \text{balance} * 1.05} (\sigma_{\text{balance} \leq 10000}(\text{account}))$$

6 考试重点与综合技巧

6.1 关系代数表达式书写要点

1. 先缩小，后连接：尽量先用 σ 过滤，减少参与笛卡尔积/连接的元组数，提高效率。
2. 同名属性处理：笛卡尔积中若有同名属性，必须用 ρ 重命名或使用带点号前缀的形式（如 $r.A$ ）区分。
3. 集合运算兼容性： $\cup$ 、 $-$ 、 $\cap$ 要求两个关系同度且对应域相容。必要时用 π 调整列数和顺序。
4. 去重自动发生： π 、 $\cup$ 、 $\cap$ 自动去重； σ 、 $\times$ 、 $\bowtie$ 、 $-$ 不去重（但关系本就不应有重复元组）。
5. ”所有/全部”语义：优先考虑除法 $\div$ 或双重否定（差运算）。 [p.60]
6. 自然连接 vs θ -连接：自然连接 $\bowtie$ 自动在所有同名属性上做等值连接并去重列，是最常用的连接。
 θ -连接（条件连接）可自定义非等值条件。

6.2 关键等价转换（考点）

- $r \cap s = r - (r - s)$ [p.56]
- 内连接 = $\sigma_{\text{条件}}(r \times s)$ 或自然连接 $r \bowtie s$
- $\sigma_{p_1 \wedge p_2}(r) = \sigma_{p_1}(\sigma_{p_2}(r))$ （选择可分解）
- $\pi_A(\sigma_p(r))$ 与 $\sigma_p(\pi_A(r))$ 通常不等价（投影可能丢失选择条件所需的属性） [p.52]
- 外连接 = 内连接 $\cup$ 未匹配元组（补 null）

6.3 常考关系模式（银行示例）

```
branch    (branch_name, branch_city, assets)
customer  (customer_name, customer_street, customer_city)
account   (account_number, branch_name, balance)
loan      (loan_number, branch_name, amount)
depositor (customer_name, account_number)
borrower  (customer_name, loan_number)
```

[p.43]

6.4 常考关系模式（学校示例）

```
student s  (学号, 姓名, 性别, 年龄)
course c   (课程号, 课程名, 学分)
sc         (学号, 课程号, 成绩)
```

[p.68–69]

6.5 典型考题类型

1. 基本查询：给定条件，写关系代数表达式（选择 + 投影组合）。
2. 多表查询：需连接两个及以上关系（通常用 $\bowtie$ 或 $\sigma(\times)$ 组合）。
3. ”至少/全部”查询：需要除法 $\div$ 或双重否定。
4. 最大值查询：用更名 + 笛卡尔积 + 差运算（如找最大余额）。 [p.47]

5. ”非”查询：用差运算 $-$ 。
6. 聚集与分组：用 G 写分组统计表达式。
7. 表达式等价判断：判断两个表达式是否等价，特别要注意投影和选择的顺序。 [p.52]
8. 修改操作：正确使用 $\leftarrow$ 与 $\cup/-$ 表达 insert/delete/update。

7 附录：符号速查表

符号	英文名	中文名
$\sigma_p(r)$	Selection	选择（按条件选行）
$\pi_A(r)$	Projection	投影（选列，去重）
$r \cup s$	Union	并（同度 + 相容域）
$r - s$	Set Difference	集合差（同度 + 相容域）
$r \times s$	Cartesian Product	笛卡尔积/广义积
$\rho_x(E)$	Rename	更名/重命名
$r \cap s$	Intersection	交
$r \bowtie s$	Natural Join	自然连接
$\bowtie_\theta$	Theta Join	θ -连接/条件连接
$\bowtie_l ()$	Left Outer Join	左外连接
$\bowtie_r ()$	Right Outer Join	右外连接
$\bowtie_f ()$	Full Outer Join	全外连接
$r \div s$	Division	除（"for all" 语义）
$r \leftarrow E$	Assignment	赋值
$\mathcal{G}$	Aggregation	聚集函数

7.1 完整 SQL 连接类型速查

连接类型	说明
INNER JOIN	内连接：只返回匹配行
LEFT OUTER JOIN	左外连接：保留左表所有行
RIGHT OUTER JOIN	右外连接：保留右表所有行
FULL OUTER JOIN	全外连接：保留两表所有行
NATURAL JOIN	自然连接：自动匹配同名列
CROSS JOIN	交叉连接（笛卡尔积）

——祝考试顺利——